Informe de QA — Módulo de Pedidos

Qué tenía el sistema, cómo lo revisé, qué encontré y qué mejoré

Autor: Ing. Edgar Rojas Apaza · **Fecha:** 4 de agosto de 2026
Sistema revisado: ABDFacturacion (Java EE / JSF) · **Sistema nuevo:** pedidos-react
Base de datos: Oracle 12c · PDBABD · **Repositorio:** github.com/edgar2420/FacturacionABD · rama `main`

1. Por qué hice este trabajo y hasta dónde llegué

Me pidieron migrar el módulo de Pedidos a una interfaz moderna. Antes de tocar nada quise saber exactamente qué tenía entre manos: dónde estaba la lógica, qué funcionaba bien y qué no. Este informe es el resultado de esa revisión y de lo que construí después.

Lo que sí revisé: el módulo de Pedidos completo —búsqueda de clientes, vendedores y productos, creación del pedido, manejo de lotes, cierre y pago— más la parte de la base de datos que ese módulo usa.

Lo que no revisé: Facturas, Recibos, PreFacturas, MisPedidos ni los reportes de Jasper. No entraban en el encargo. Tampoco modifiqué ni una línea del proyecto Java original ni de la base de datos.

> **Regla que me puse desde el principio.** El sistema apunta a la base de **producción** (`192.168.0.5/PDBABD`). Así que me impuse tres cosas y las cumplí:
>
> 1. Contra producción, **solo consultas de lectura**.
> 2. Cualquier prueba que necesite escribir va a **tablas de prueba** aparte.
> 3. Todo lo que afirme en este informe tiene que estar **comprobado**, no supuesto. Y si algo no lo pude comprobar, lo digo.

2. Cómo hice el QA, paso a paso

Esta parte es la que más me interesa dejar escrita, porque cualquiera del equipo puede repetirla y llegar a lo mismo. No es magia: son cuatro frentes.

> ### Paso 1 — Leer el código Java para entender qué hace realmente
>
> Empecé por los DAO y los beans, que es donde está el flujo: `src/java/dao/PedidosDao.java` y `src/java/beans/PedidosBean.java`.
>
> Lo primero que descubrí me ahorró semanas: **la lógica de negocio no está en Java**. Los DAO solo abren la conexión, llaman a una función de Oracle y traducen el resultado. Toda la inteligencia vive en el paquete `ABD.PCK_OPERACIONES`.
>
> Eso cambió mi enfoque por completo: reescribir la pantalla no cambia las reglas. La aplicación nueva tenía que llamar a **las mismas funciones**, no reimplementarlas.

> ### Paso 2 — Leer el código PL/SQL, que es donde de verdad pasan las cosas
>
> El código fuente de un paquete de Oracle se puede leer desde la propia base, sin necesidad de que nadie te pase archivos. Esta es la consulta:

```
SELECT LINE, TEXT
  FROM ALL_SOURCE
 WHERE OWNER = 'ABD'
   AND NAME  = 'PCK_OPERACIONES'
   AND TYPE  = 'PACKAGE BODY'
 ORDER BY LINE;
```

Con eso leí una por una las funciones que usa Pedidos. Aquí salieron la mitad de los defectos de este informe. Es tedioso pero no hay atajo: son las líneas que realmente insertan, borran y calculan.

Un detalle que conviene saber: **hay funciones repetidas con distinto número de parámetros**. `F_OBTENER_PRODUCTOS` tiene dos versiones, una de 3 parámetros y otra de 5. Si uno lee solo la primera que aparece, saca conclusiones equivocadas. Hay que buscar todas.

Paso 3 — Preguntarle a los datos, no a la documentación

Este fue el paso más útil. En vez de suponer cómo se usa el sistema, se lo pregunté a los 16.547 pedidos que ya existen.

Ejemplo 1 — ¿de verdad cada usuario trabaja con un solo almacén?

```
SELECT CASE WHEN u.UNI_NEG = p.ALMACEN THEN 'coincide' ELSE 'DISTINTO' END situacion,
       COUNT(*) pedidos
  FROM IBSVENTA.PEDIDOS p
  LEFT JOIN ABD.USUARIOS u ON u.LOGIN = p.LOGIN
 GROUP BY CASE WHEN u.UNI_NEG = p.ALMACEN THEN 'coincide' ELSE 'DISTINTO' END;
```

Resultado: 16.544 coinciden, 3 son de usuarios ya dados de baja. Ni uno distinto. La regla se cumple sola desde hace años.

Ejemplo 2 — ¿a qué ciudad le vende cada almacén? Iba a asumir que `AL-VEN-ABD-FAB` era La Paz, porque «FAB» es la fábrica. Menos mal que lo comprobé:

```
SELECT p.ALMACEN, pe.CIUDAD, COUNT(*) pedidos
  FROM IBSVENTA.PEDIDOS p
  JOIN ABD.PERSONAS pe ON pe.CODIGO = p.CLIENTE
 GROUP BY p.ALMACEN, pe.CIUDAD
 ORDER BY p.ALMACEN, COUNT(*) DESC;
```

El 90 % de los 16.385 pedidos de FAB fueron a clientes de **Santa Cruz**, no de La Paz. Si me hubiera fiado del nombre, todo lo que construí encima habría estado mal.

Lección: los nombres de los códigos mienten; el histórico no.

Paso 4 — Escribir pruebas automáticas, en tres niveles

El sistema original no tenía ninguna prueba. Armé 124, en tres niveles, y cada nivel sirve para algo distinto:

Nivel	Nº	Para qué sirve	Necesita base
Unitarias *node:test*	51	Las reglas puras: el reparto de lotes, las fechas, las cantidades, el reconocimiento de ciudades. Corren en menos de un segundo, así que las lanzo mientras programo.	No
De interfaz *Cypress*	63	Abren la aplicación en un navegador de verdad y hacen lo que haría un vendedor: buscar, tocar,	No, va simulada

Nivel	Nº	Para qué sirve	Necesita base
		escribir, confirmar.	
De integración *script propio*	10	Contra Oracle real, con lotes y saldos de verdad, para confirmar que lo que se graba es lo que tiene que grabarse.	Sí, en tablas de prueba

Por qué las de Cypress no tocan la base. Esto no es comodidad, es obligación: si las pruebas hablaran con producción, cada vez que las corriera estaría creando pedidos reales. Así que intercepto todas las llamadas a la API y respondo con datos de prueba. La aplicación no nota la diferencia.

Cómo escribí las pruebas para que duren. Busco por **texto visible** —«Buscar cliente», «Elegir vendedor»— y no por clases de CSS. El diseño de esta aplicación lo cambié cinco veces; si las pruebas dependieran del CSS, se romperían en cada cambio y acabaría ignorándolas. Buscando por texto solo fallan cuando de verdad se rompe algo.

Paso 5 — Probar lo que escribe, sin escribir en producción

Me quedaba lo más delicado: comprobar que el reparto en lotes graba bien. Eso obliga a escribir, y escribir en producción era justo lo que no quería.

Me pasaron dos tablas de prueba, `IBSVENTA.PPEDIDOS` y `PLIN_PEDIDOS`. Lo primero fue verificar que sirvieran:

```
SELECT COLUMN_NAME, DATA_TYPE, DATA_LENGTH
  FROM ALL_TAB_COLUMNS
 WHERE OWNER = 'IBSVENTA' AND TABLE_NAME IN ('PPEDIDOS','PLIN_PEDIDOS')
 ORDER BY TABLE_NAME, COLUMN_ID;
```

Son copia exacta: 32 y 23 columnas, mismos tipos, ninguna de más ni de menos. Pero encontré el problema: **ningún código de la base las usa**. El paquete sigue insertando en las tablas reales, y eso no se puede redirigir desde afuera porque el `INSERT` está dentro del PL/SQL.

La solución fue un **modo prueba** en el servidor (`MODO_PRUEBA=1`): cuando está activo, la aplicación escribe directo en las tablas de prueba en vez de llamar al paquete. Los catálogos, saldos y lotes se siguen leyendo de producción, que es justo lo que quiero para probar con datos de verdad. **Solo cambia dónde se escribe.**

El precio sí es el real: llamo a `IBSADM.GET_IMPORTE`, la misma función que usa el paquete.

Paso 6 — Comprobar que no ensucié nada

Después de cada prueba que escribe, verifico. No basta con confiar:

```
SELECT NUMERO, LOGIN, ALMACEN, TO_CHAR(FECHA,'DD/MM HH24:MI') cuando, ESTADO
  FROM IBSVENTA.PEDIDOS
 WHERE FECHA >= TRUNC(SYSDATE)
 ORDER BY FECHA;
```

Los 9 pedidos del día eran de `jcoulthad`, `ecespedes`, `jrevollo` y `rriveros` —vendedores reales—. Ninguno mío. Y las tablas de prueba quedaron en cero, porque el script se limpia solo al terminar.

3. Qué me encontré

3.1 La tecnología

Plataforma	Java 8, Java EE 7 Web
Interfaz	JSF 2.2 (Facelets) con PrimeFaces 6.0, renderkit PRIMEFACES_MOBILE
Persistencia	Hibernate 4.3.10 con mapeos XML
Base	Oracle 12c — esquemas ABD, IBSVENTA, IBSADM, IBSPLA
Informes	JasperReports 5.6, iText, PDFBox, ZXing para los QR
Servidor	Payara 5 / GlassFish
Compilación	Ant + NetBeans, sin Maven. Los JAR van por ruta absoluta, así que el proyecto solo compila en la máquina donde se armó

3.2 Dónde vive la lógica

En Oracle, no en Java. Estas son las funciones del paquete que usa Pedidos:

Función	Qué hace
`F_VALIDA_USUARIO`	Comprueba usuario y contraseña
`F_OBTENER_PRODUCTOS`	Catálogo con precio y saldo, **una fila por lote**
`F_CREAR_PEDIDO`	Crea la cabecera y la primera línea juntas
`F_CREAR_DETALLE_PEDIDO`	Añade una línea más
`F_ELIMINAR_DETALLE_PEDIDO`	Borra líneas **por código de ítem**
`F_TERMINAR_PEDIDO`	Totaliza y pasa el estado a «V»
`F_PAGAR_PEDIDO`	Registra forma de pago y datos de factura

3.3 Los almacenes

Todo gira alrededor del almacén, que sale de `ABD.USUARIOS.UNI_NEG`. Hay ocho:

Almacén	Usuarios	Productos con saldo	Le vende a
`AL-VEN-ABD-FAB`	87	110	Santa Cruz (90 % del histórico)
`AL-SUM-ABD-FAB`	31	0	Suministros de planta, no vende
`AL-VEN-ABD-LPZ`	29	96	La Paz
`AL-VEN-ABD-CB1`	27	111	Cochabamba (80 %)
`AL-VEN-ABD-SUC`	8	93	Sucre (95 %)
`AL-VEN-ABD-SC8`	3	0	Santa Cruz, sin stock cargado
`AL-VEN-ABD-ORU`	3	83	Oruro

Almacén	Usuarios	Productos con saldo	Le vende a
AL-VEN-ABD-ARG	1	0	Argentina

Buena noticia: el aislamiento por almacén funciona y siempre funcionó. Lo comprobé con la consulta del Paso 3. Nadie tiene cuenta en dos almacenes y nadie grabó nunca en dos. **Vender existencias de un almacén desde otro ya era imposible** antes de que yo tocara nada.

3.4 El problema de los lotes, que era el encargo de fondo

El catálogo devolvía **una fila por cada lote**. En el almacén de Fábrica: 233 filas para solo 110 productos. Sesenta de esos productos tienen más de un lote.

El vendedor tenía que elegir el lote a mano y solo veía el saldo de ese lote. Con el producto PT982:

Lote	Vence	Saldo
F244	30/09/26	5
F245	30/09/26	120
F250	31/12/26	200
Había de verdad		**325**

Si el cliente pedía 300, el vendedor veía tres filas y ninguna llegaba a 300. **Parecía que no había stock, habiendo de sobra.** Ese era el problema a resolver.

4. Los defectos que encontré

Son once. De cada uno digo **cómo lo detecté** y **cómo comprobarlo**, para que nadie tenga que creerme sin más.

4.1 Graves

GRAVE ABIERTO **D-01 · La contraseña no se comprueba si el usuario no tiene clave guardada**

Dónde: `ABD.PCK_OPERACIONES.F_VALIDA_USUARIO`

Cómo lo encontré: leyendo el cuerpo de la función. Cuando el usuario no tiene fila en la tabla de claves, la variable interna queda en NULL. Y en Oracle, comparar contra NULL no da verdadero ni falso: da **desconocido**. El `IF` que debería rechazar el acceso nunca se cumple, y la función termina devolviendo `'1'`, o sea «adelante», **con cualquier contraseña**.

Afecta también al sistema JSF que está en producción hoy, no solo al nuevo.

No lo corregí porque está en la base y no debía tocarla. Lo dejo levantado para quien la administra.

GRAVE CORREGIDO **D-02 · Inyección SQL en la búsqueda de clientes**

Dónde: `PedidosDao.buscarCliente()`, línea 125

```
sql = "from Personas where upper(nombre) like '%"+valor.toUpperCase()+"%' and codigoCliente='SI'
order by nombre";
```

Cómo lo encontré: a simple vista. Lo que escribe el usuario se pega dentro de la consulta. Una comilla simple basta para alterarla. El mismo patrón se repite en varias consultas del archivo.

Cómo lo corregí: en la aplicación nueva uso variables de vínculo (*bind variables*). El valor viaja aparte de la consulta, así que por mucho que el usuario escriba comillas, Oracle lo trata como texto y nunca como instrucción. El código Java original lo dejé intacto.

GRAVE ABIERTO **D-03 · El precio que se muestra no es el que se graba**

Cómo lo encontré: comparando las dos funciones. La que arma la pantalla pide el precio con la **fecha de entrega**:

```
IBSADM.get_precio('P', i.codigo, p_cliente, p_almacen, to_date(p_fecha,'dd/mm/yyyy'))
```

Pero la que graba la línea lo calcula con **la fecha de hoy**:

```
v_fecha  DATE := SYSDATE;
v_precio := IBSADM.GET_IMPORTE('P', p_item, p_cliente, p_almacen, v_fecha, 'BS', p_cantidad,
v_dcto);
```

Cómo comprobarlo (producto PT982, cliente I00060):

```
SELECT IBSADM.GET_PRECIO('P','PT982','I00060','AL-VEN-ABD-FAB',
                         TO_DATE('04/09/2026','dd/mm/yyyy')) * (-1) precio_pantalla,
       IBSADM.GET_IMPORTE('P','PT982','I00060','AL-VEN-ABD-FAB',
                          SYSDATE,'BS',1,0) precio_grabado
  FROM DUAL;
```

Cálculo	Fecha que usa	Precio
Lo que ve el vendedor	04/09/2026, la entrega	Bs 50,00
Lo que queda grabado	hoy	**Bs 45,00**

Un **10 % de diferencia** en este producto. Pasa siempre que la fecha de entrega caiga en un tramo de precio distinto al de hoy.

No lo pude corregir: el cálculo está dentro del PL/SQL, y como la aplicación nueva llama al mismo procedimiento, **hereda el defecto**. Arreglarlo exige cambiar el paquete.

4.2 Medios

MEDIO CORREGIDO **D-04 · Búsquedas sin límite**

Cómo lo encontré: buscando clientes por la letra «A» y mirando lo que volvía. **7.108 registros** en una sola respuesta, todos a la pantalla. Ninguna consulta tenía paginación.

Cómo lo corregí: 15 por página, tope de 50. Con los productos tuve que hilar más fino: vienen de un cursor y hay que **agruparlos por producto antes de paginar**, porque si no los lotes de un mismo producto quedan partidos entre dos páginas y el saldo sale incompleto.

MEDIO CORREGIDO **D-05 · Llamada mal escrita al eliminar una línea**

Dónde: `PedidosDao.eliminarDetallePedido()`, línea 671

```
conn.prepareCall("{? = call ABD.PCK_OPERACIONES.F_ELIMINAR_DETALLE_PEDIDO(?,?,?,?)");
```

Cómo lo encontré: comparando esta llamada con las demás del mismo archivo. Falta la llave de cierre `}`. Y de paso, el mensaje del `catch` dice `F_ELIMINARAR_DETALLE_PEDIDO`, con una sílaba de más.

Cómo lo corregí: en la aplicación nueva la llamada va bien formada.

MEDIO ABIERTO **D-06 · Credenciales en texto plano dentro del repositorio**

`hibernate.cfg.xml` y los `constantes*.properties` llevan el usuario y la contraseña de Oracle escritos tal cual. El `web.xml` lleva las claves de reCAPTCHA.

No lo corregí porque no debía tocar el proyecto Java. Pero si el repositorio se hace público o se comparte con alguien de fuera, **esas credenciales hay que rotarlas antes**. En mi aplicación las saqué a un archivo `.env` que está excluido del control de versiones.

MEDIO ABIERTO **D-07 · No existe relación entre cliente y almacén**

Cómo lo encontré: buscando una columna que ligara al cliente con un almacén. No hay ninguna: `ABD.PERSONAS` solo tiene `CIUDAD`, y es texto libre. El sistema JSF tampoco filtra: su consulta no menciona almacén ni ciudad.

Pensé en deducirlo de la ciudad, hasta que miré cómo está escrita:

```
SANTA CRUZ · SANTA RUZ · SAANTA CRUZ · SANTA CRU · SABTA CRUZ
SNTA CRUZ · SANTA CRUZZ · SANTAC RUZ · SANT CRUZ
SANTA CRUZ-MONTERO · SANTA CRUZ-WARNES · SATELITE · EL ALTO
```

Todo eso es la misma ciudad escrita de nueve maneras, más localidades que pertenecen al departamento.

Qué hice: avisar, no bloquear. La aplicación marca al cliente que parece de otra región y lo explica al elegirlo, pero deja continuar. Bloquear con estos datos dejaría fuera clientes buenos, y un aviso falso enseña a ignorar los avisos. La solución de fondo es una columna de almacén en `PERSONAS`, y eso ya toca la base.

4.3 Latentes — hoy no molestan, mañana sí

MEDIO **LATENTE** **D-08 · El almacén se compara con LIKE en vez de con igualdad**

Dónde: `F_OBTENER_PRODUCTOS`

```
and s.codigo_almacen LIKE '%'||p_almacen||'%'
```

Cómo lo encontré: leyendo la consulta me llamó la atención el `LIKE`, y fui a ver si había almacenes que fueran subcadena de otros:

```
SELECT CODIGO_ALMACEN, COUNT(*) filas, SUM(SALDO_ACTUAL) saldo
  FROM IBSVENTA.SALDOS_ITEMS
 WHERE CODIGO_ALMACEN LIKE '%AL-VEN-ABD-CB1%'
 GROUP BY CODIGO_ALMACEN;
```

Almacén que casa	Filas	Saldo
AL-VEN-ABD-CB1	3.087	171.756
.AL-VEN-ABD-CB1	8	0
3AL-VEN-ABD-CB1	2	0

Hay almacenes con un punto y con un número delante. Pedir Cochabamba devuelve los tres.

Hoy no da números malos porque esos dos tienen saldo cero y el filtro los descarta. Pero el día que alguien cargue existencias en `.AL-VEN-ABD-CB1`, se van a sumar al catálogo de Cochabamba **sin que nadie se entere**. Lo mismo pasa con FAB, LPZ y ARG.

MEDIO **LATENTE** **D-09 · El saldo que se muestra no descuenta lo pendiente de entrega**

La consulta filtra por `HAVING SUM(saldo_actual − pendiente_entrega) > 0` pero **devuelve** `SUM(saldo_actual)`, sin restar. O sea que el número que se enseña es mayor que lo realmente disponible.

Cómo comprobé si importa:

```
SELECT COUNT(*) filas,
       SUM(CASE WHEN NVL(PENDIENTE_ENTREGA,0) > 0 THEN 1 ELSE 0 END) con_pendiente
  FROM IBSVENTA.SALDOS_ITEMS
 WHERE CODIGO_ALMACEN = 'AL-VEN-ABD-FAB';
```

De las 14.749 filas de stock, **ninguna** tiene `pendiente_entrega` distinto de cero. La columna sencillamente no se usa, así que hoy no hay error. Lo dejo anotado por si algún día se empieza a usar: ahí sí se podría vender por encima de lo que hay.

4.4 Leves

LEVE CORREGIDO **D-10 · Etiquetas de formulario sin asociar a su campo**

Cómo lo encontré: escribiendo las pruebas de Cypress. Al intentar seleccionar un campo por su etiqueta me di cuenta de que no estaban ligados. Un lector de pantalla leía la caja de texto sin decir para qué era, y tocar la etiqueta en el celular no daba el foco al campo.

Cómo lo corregí: con `id` y `htmlFor`. Es un detalle chico, pero lo encontré **gracias a escribir las pruebas**, que es justo para lo que sirven.

LEVE ABIERTO **D-11 · Eliminar una línea borra todas las del mismo producto**

`F_ELIMINAR_DETALLE_PEDIDO` borra con `WHERE codigo_item = ...`, sin el número de línea. Si un producto ocupa varias líneas, se van todas.

Con el reparto por lotes esto en realidad conviene —el vendedor quita «el producto», no «un lote»— pero hay que saberlo, porque no existe forma de borrar una sola línea.

5. Qué mejoré

5.1 Lotes sumados y reparto automático

Es el cambio de fondo, el que me pidieron.

ANTES

Tres filas del mismo producto, saldos 5, 120 y 200.
El vendedor elige el lote a mano.
Pidiendo 300, parece que no hay stock.

AHORA

Una sola fila, saldo 325, con la marca «3 lotes».
El vendedor solo escribe la cantidad.
El sistema reparte solo.

La regla de despacho es FEFO: sale antes lo que caduca antes.

1. Primero el lote que **vence antes**.
2. Si dos vencen el mismo día, primero el de **menor saldo**, para liquidar los restos.
3. Un lote sin fecha legible va al final: no puedo afirmar que venza pronto, y sacarlo primero sería arriesgado.

Pidiendo 300 unidades de PT982 queda así:

Lote	Vence	Saldo	Sale	Por qué
F244	30/09/26	5	**5**	vence antes, y de los dos del 30/09 es el más chico
F245	30/09/26	120	**120**	mismo vencimiento, más saldo
F250	31/12/26	200	**175**	vence después, se usa al final

Cuatro decisiones que tomé y por qué:

- **El servidor vuelve a leer el producto al confirmar.** No me fío de lo que manda el navegador: entre que el vendedor busca y le da a guardar pueden pasar minutos, y otro vendedor puede haberse llevado el lote. El reparto se calcula con el saldo de ese instante.
- **Cada lote es una línea del pedido**, porque cada fila de `LIN_PEDIDOS` guarda un solo lote. Pedir 300 genera tres líneas, y el vendedor las ve así.
- **Si el saldo total no alcanza, no reparto nada** y aviso cuánto falta. Prefiero eso a dejar el pedido a medias.
- **El vendedor ve el reparto antes de confirmar**, mientras escribe la cantidad. No quería una caja negra que decide sola sin explicar.

Una limitación que no pude evitar. Los procedimientos de Oracle hacen `COMMIT` dentro de sí mismos, así que un reparto en varios lotes **no se puede envolver en una transacción**. Si un tramo falla a mitad, lo compenso borrando las líneas del producto —y solo si el producto no estaba ya en el pedido, porque si estaba me llevaría líneas buenas—. Cuando no se puede compensar, informo el estado real y que decida la persona.

5.2 La interfaz

Aspecto	Antes	Ahora
Elegir cliente o producto	seleccionar arriba, bajar y pulsar un botón al pie	se toca la fila, se abre su ficha y se confirma ahí mismo
Tablas en celular	se cortaban, había que desplazar de lado	se convierten en tarjetas con etiqueta y valor
Fecha de entrega	se escribía a mano	calendario del propio teléfono, con hoy como mínimo, más atajos «Hoy» y «Mañana»
Avisos	mensajes fáciles de pasar por alto	los errores en modal centrado; los éxitos en aviso que se va solo
Tema	único	arranca siempre en claro; el oscuro se recuerda si se elige
Almacén de trabajo	código en letra chica, solo en pantalla grande	nombre legible siempre visible, también en celular
Iconos	Font Awesome	SVG propios, sin librería externa y sin emojis
Instalación	solo navegador	instalable como aplicación (PWA), abre sin conexión

Sobre la PWA: los navegadores solo permiten instalar por HTTPS o desde `Localhost`. Para ponerla en los celulares de los vendedores va a hacer falta servirla con certificado.

5.3 Seguridad

- **Variables de vínculo** en todas las consultas: cierra la inyección SQL.
- **El almacén nunca llega del navegador.** Sale de `UNI_NEG` al iniciar sesión y vive en la sesión del servidor. Aunque alguien manipule la petición, no puede pedir stock de otro almacén: no hay forma de nombrarlo.
- **Las credenciales salen del código** y pasan a un `.env` excluido del repositorio.
- **Valido también en el servidor.** Lo que valida el navegador es comodidad; la validación que manda es la de atrás.
- **Los errores de Oracle no llegan al navegador.** Se registran en el servidor y al usuario le llega un mensaje neutro, sin trazas.

6. Cómo repetir este QA

Todo lo de este informe se puede volver a correr. Estos son los comandos:

```
# 51 pruebas de reglas. No necesitan base de datos, tardan un segundo.
cd pedidos-react/server && npm test

# Levantar la interfaz
cd pedidos-react/web && npm run dev

# 63 pruebas de interfaz en navegador real. No tocan la base.
cd pedidos-react/qa && npm run qa
```

```
# 10 comprobaciones contra Oracle, en tablas de prueba.
# Requiere MODO_PRUEBA=1 en el .env
cd pedidos-react/server && npm run prueba:reparto
```

La última es la que da más confianza, porque busca sola un producto real con varios lotes, pide una cantidad que obligue a partirlo y comprueba diez cosas:

```
OK  hay una linea por cada lote del reparto
OK  cada linea lleva su lote y su cantidad
OK  la suma de las lineas es la cantidad pedida
OK  las lineas van numeradas 1, 2, 3...
OK  el primer lote es el que vence antes
OK  el precio lo calculo Oracle, no viene en cero
OK  la descripcion del producto se resuelve
OK  al terminar, el total de la cabecera es la suma de las lineas
OK  al terminar, el estado pasa a V
OK  eliminar el producto borra sus lineas de todos los lotes
```

Si Cypress no arranca y dice bad option: --smoke-test, no es el programa: es la variable de entorno ELECTRON_RUN_AS_NODE=1, que la ponen los editores que a su vez son Electron y hace que Cypress arranque como si fuera Node. Se quita con Remove-Item Env:\ELECTRON_RUN_AS_NODE. Reinstalar el binario no sirve de nada; lo comprobé perdiendo un rato.

7. Qué falta y qué recomiendo

7.1 Lo que no pude probar

El paquete `ABD.PCK_OPERACIONES` no queda ejercitado por la prueba de integración: sus `INSERT` están dentro del PL/SQL y apuntan a las tablas reales, así que no hay manera de redirigirlos desde afuera. Lo replico —incluido el borrado por código de ítem— y el precio sí sale de la función real.

Lo que sí quedó probado de punta a punta es **el reparto en lotes**, que es lo que no existía. El paquete lleva años funcionando; lo nuevo es lo mío, y eso está cubierto.

Para cubrir también el paquete haría falta una copia suya que escriba en las tablas de prueba —ya existe `PCK_OPERACIONES_BACK`, así que el equipo sabe duplicarlo—. Eso lo tiene que hacer quien administra la base.

7.2 Recomendaciones, por orden de urgencia

#	Defecto	Qué hacer	Quién
1	D-01	Corregir `F_VALIDA_USUARIO`: usar `NVL` o `IS NULL` para que una clave ausente **deniegue** el acceso en vez de concederlo	Administrador de la base
2	D-03	Unificar la fecha del precio: `F_CREAR_DETALLE_PEDIDO` debería recibir la fecha de entrega, no usar `SYSDATE`	Administrador de la base
3	D-06	Rotar credenciales de Oracle y claves de reCAPTCHA si el repositorio se comparte	Sistemas
4	D-08	Cambiar `LIKE` por `=` en el filtro de almacén, o limpiar los almacenes con punto y número delante	Administrador de la base
5	D-07	Valorar una columna de almacén en `PERSONAS` si se quiere bloquear de verdad la venta entre departamentos	Decisión de negocio

#	Defecto	Qué hacer	Quién
6	D-09	Si se empieza a usar `pendiente_entrega`, restarlo también en el saldo que se muestra	Administrador de la base
7	—	Revisar `server/src/zonas.js`: los almacenes LPZ, ORU, SC8 y ARG no tienen histórico y su región la deduje del sufijo. Si alguna está mal, se corrige ahí y en un solo sitio	Negocio
8	—	Servir la aplicación por HTTPS para poder instalarla en los celulares	Sistemas

7.3 Advertencias para quien tome esto después

El entorno apunta a producción. El `.env` tiene `192.168.0.5/PDBABD`. Cualquier pedido creado desde la aplicación es un pedido real, visible en el sistema JSF.

Para probar sin riesgo, poner `MODO_PRUEBA=1`. Los pedidos se graban en las tablas de prueba y los catálogos se siguen leyendo de producción. **Dejar vacío en cualquier despliegue real.**

`ALMACEN_DEV` es solo para desarrollo: permite entrar con cuentas que no están en `ABD.USUARIOS` asignándoles un almacén fijo. También debe quedar vacía en producción.

8. Resumen

Punto	Antes	Ahora
Filas por producto en la búsqueda	1 por lote	1 por producto
Saldo que ve el vendedor	el de un lote suelto	la suma de todos
Elección de lote al vender	manual	automática (FEFO)
Paginación	ninguna	15 por página
Clientes devueltos en una búsqueda	hasta 7.108	15
Pruebas automáticas	0	124
Consultas con inyección SQL	sí	no
Uso en celular	tabla cortada	tarjetas

Once defectos encontrados: cinco corregidos en la aplicación nueva, cuatro siguen abiertos porque están en la base o en el sistema JSF y no debía tocarlos, y dos son latentes: hoy no hacen daño, pero lo harán en cuanto cambien los datos.

Elaborado por el Ing. Edgar Rojas Apaza. Todo lo afirmado está verificado contra la base de datos Oracle de producción —únicamente con consultas de lectura— o contra el código fuente de ambos sistemas. Los saldos, lotes, precios y pedidos citados son reales y estaban vigentes el 4 de agosto de 2026. Ninguna comprobación modificó datos de producción.